

Code Assessment of the CHFD Smart Contracts

PUBLIC

Date [June 24, 2026](#)

Produced for



SWISS

Stablecoin AG

By



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	5
4	Trust Model	9
5	System Considerations	13
6	Terminology	15
7	Findings	16
8	Limitations and use of report	25

1 Executive Summary

Dear Swiss Stablecoin Team,

Thank you for trusting ChainSecurity with the review of the CHFD stablecoin contracts.

Trusted globally to secure tier-one digital assets, ChainSecurity provides the specialized, independent technical auditing required for regulated, compliance-first stablecoin systems.

We greatly appreciate the opportunity to collaborate with Swiss Stablecoin on bringing a fully regulated Swiss-Franc stablecoin to Ethereum.

This executive summary provides a high-level overview of the subjects covered in our review of CHFD and provides a distilled analysis of its security posture, to assist in forming an opinion on its risk profile.

Swiss Stablecoin implements CHFD, a Swiss-Franc-denominated stablecoin issued on Ethereum through two upgradeable contracts. CHFD is the ERC-20 token that holders own and transfer, while CHFD_VASP is the compliance layer holding VASP registration, holder allowlisting, limits and the transfer-validation logic.

We have conducted a thorough review of the contracts and identified findings across three main areas. Transfer validation is the single gate every mint, burn, and transfer must pass, and we found that the enforcement burn paths cannot act on the sanctioned or frozen holders they target, see [Burn Paths Require an Active Holder and VASP and an Unpaused Token](#). Signature handling secures all VASP administration and the gasless burn flow. Replay protection holds, but a signer cannot invalidate a signature before its deadline passes, see [Signers Cannot Cancel Issued Signatures Prior to Deadline Expiry](#). Access control concentrates the system's authority in a few privileged roles with no on-chain recovery levers, see [Role Concentration and Lack of Recovery Levers](#). The reported issues have been addressed in the latest version of the codebase.

The system as implemented is described in our [System Overview](#), alongside which we record system considerations inherent to the design of CHFD that bear on integration and safe operation. VASP admins must be EOAs, so smart-contract wallets such as Safe cannot sign, see [Signature Verification Accepts EOA Signers Only](#). Also, state is held in sequential rather than namespaced storage, which constrains how the contracts may be upgraded, see [Custom State Uses Sequential Storage Rather than ERC-7201](#).

In summary, we find that the codebase provides a good level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the CHFD repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	16 June 2026	b1009b6da190d25f4c32fd905e5e67a9867cf6ff	Initial review (private commit)
2	22 June 2026	011878cc8dcd258cd129bdc6e68c7ecad276fbb0	Fixes

For the Solidity smart contracts, the compiler version `0.8.34` was chosen.

The following files are in scope:

```
src/
├── CHFD.sol
├── CHFD_V2.sol
├── CHFD_VASP.sol
├── CHFD_VASP_V2.sol
├── Errors.sol
├── Events.sol
└── Statuses.sol
```

2.1.1 Excluded from scope

Any file not explicitly cited above is out of scope.

All external libraries and third-party dependencies are excluded from scope.

The off-chain components of the system are out of scope. This review covers the on-chain Solidity contracts only. The backend services, relay, key management, identity, indexer, or frontend that operate them are not reviewed.

3 System Overview

This system overview describes the initially received version (**Version 1**) of the contracts as defined in the [Assessment Overview](#).

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Swiss Stablecoin offers CHFD, a Swiss-Franc-denominated stablecoin issued on Ethereum through two upgradeable contracts. CHFD is the token itself, the ERC-20 that holders own and transfer. CHFD_VASP is the compliance layer. It holds VASP registration, holder allowlisting, limits, and status. It defines the transfer validation logic by CHFD. The split keeps the token close to the ERC-20 standard while the compliance logic evolves independently.

3.1 CHFD

CHFD is the token layer, an upgradeable ERC-20 with permit, pausing, and role-based access control. It has 6 decimals, the name "Swiss Stablecoin", and the symbol "CHFD". `initialize()` sets the name and symbol, grants the deployer the token roles, and records the address of the compliance contract, which the admin can later change through `setVaspContractAddress()`. Balances move only through the `_update()` override, which enforces the paused and compliance state check.

3.1.1 Minting

The `MINTER_ROLE`-gated `mint()` issues CHFD to a recipient. It first calls `CHFD_VASP.validateTransfer()`, such that issuance is bound to the recipient's holder limit and reverts with `ExceedsHolderLimit` when the resulting balance would exceed it. It then mints the amount and emits `MintEvent` with the recipient's `vaspId`. The recipient must be a registered, active holder under an active VASP.

3.1.2 Burning

Two paths burn CHFD. The `ENFORCEMENT_ROLE`-gated `burnFrom()` burns from any holder after calling `validateTransfer(from, 0, 0)`, which requires the source holder and its VASP to be active, and emits `BurnEvent`. The `BURNER_ROLE`-gated `burnFromWithPermit()` consumes an ERC-20 permit signed by the holder to set the burner's allowance, validates the holder through `validateTransfer(owner, 0, 0)`, spends the allowance, burns the amount, and emits `BurnEvent`. CHFD implements EIP-2612 permit through OpenZeppelin.

3.1.3 Transfers

A user transfer moves balances between two non-zero addresses through `_update()`. The override enforces the paused state, calls `CHFD_VASP.validateTransfer()`, moves the balances, and emits `CHFDTransferEvent`. Mints and burns also reach `_update()`, each with one side set to the zero address, and they perform their own `validateTransfer()` call before the balance change. The checks the validator applies are described under [Transfer validation](#).

3.1.4 Pausing

The `ENFORCEMENT_ROLE` toggles the paused state through `pause()` and `unpause()`. The paused state is enforced inside `_update()`, which every balance change routes through, so while paused all user transfers, `mint()`, `burnFrom()`, and `burnFromWithPermit()` revert. The single exception is `forceTransfer()`, which sets a flag that makes `_update()` skip the pause check. Pausing does not affect role administration or the CHFD_VASP registry.

3.1.5 Enforcement

The `ENFORCEMENT_ROLE`-gated `forceTransfer()` moves a balance between two non-zero addresses regardless of pause state, holder or VASP status, or holder limit. It sets the `_forceTransferInProgress` flag, calls the parent balance update, clears the flag, and emits `ForceTransferEvent`. The flag makes `_update()` skip both the paused check and the `validateTransfer()` call for the duration of the call.

3.2 CHFD_VASP

CHFD_VASP is the compliance layer, an upgradeable contract that holds the registry of VASPs and holders and exposes the validation that the CHFD token calls on every transfer. `initialize()` sets the EIP-712 domain (name "CHFD_VASP", version "1") and grants the deployer the compliance roles.

3.2.1 VASPs and holders

A VASP holds an admin set, an admin count, and a `VaspStatus`. A holder is a record holding the `vaspId` it belongs to, a `limit`, and a `HolderStatus`. Both statuses use the same enum values:

- `None`: the entity does not exist.
- `Active`: the entity is operational.
- `Blocked`, `Locked`, `Sanctioned`, `Frozen`: the entity exists but is not active.

A VASP and holder exists when their status is not `None`. Each holder is bound to exactly one VASP. The binding is written once, and there is no instruction to remove a holder or move it to another VASP.

3.2.2 Administration

Authority over the registry is split. On one side, the `UPDATE_VASP_ROLE` holder manages the VASP lifecycle. It is the only role that creates VASPs and the only role that changes a VASP's status. VASP admins on the other side administer only their own VASP's admin set and its holders.

`addVasp()` creates an `Active` VASP with a supplied admin, registers that admin as a holder with limit 0, and reverts if that address is already a holder of any VASP. The function is gated twice: the caller must hold `UPDATE_VASP_ROLE`, and the prospective admin's signature proves control of the supplied address.

`addVaspAdmin()` and `removeVaspAdmin()` change a VASP's admin set, capped at `MAX_VASP_ADMINS` (10) and never reduced below one admin. `addHolder()` registers a holder under an active VASP with an initial limit, and reverts if the address is already a holder. `setHolderLimit()` and `setHolderStatus()` change a holder's limit and status, and require the holder to belong to the named VASP and that VASP to be active. `setVaspStatus()` sets a VASP's status to any value other than `None`.

Each VASP admin operation has a direct form and a signature form. The direct forms (`addVaspAdmin()`, `removeVaspAdmin()`, `addHolder()`, `setHolderLimit()`, `setHolderStatus()`) require `msg.sender` to be an admin of the named VASP. The signature forms (the same names suffixed `WithSig()`) are submitted by `UPDATE_VASP_ROLE` and carry a VASP admin's signature.

Adding a co-admin also registers that admin as a holder of the VASP. An address already registered as a holder of another VASP cannot be added. If the address is already a holder of the same VASP, it is promoted to admin with its existing limit and status preserved.

3.2.3 Signatures and replay protection

The signature forms and `addVasp()` are verified against the `CHFD_VASP` EIP-712 domain. Each operation has its own typed-data hash covering the operation's parameters, the signer's nonce, and a deadline. A signer holds a single `nonces` value, consumed in order on every signed operation, so a signature is single use. A signature remains valid until its deadline. The signer of a `WithSig()` operation must be a current admin of the

named VASP and the signer of `addVasp()` is the prospective admin. The `UPDATE_VASP_ROLE` holder submits the transaction in every case.

3.2.4 Transfer validation

`validateTransfer(from, to, targetAmount)` is the single compliance gate. It is `VALIDATE_TRANSFER_ROLE`-gated, so only the CHFD token contract calls it, and it is a view that reverts on a policy failure. It applies, per side:

- For a non-zero `from`: the sender must be a registered holder, the sender's VASP must be active, and the sender's holder status must be active. Otherwise it reverts with `HolderDoesNotExist`, `VaspNotActive`, or `HolderNotActive`.
- For a non-zero `to`: the receiver must be a registered holder, the receiver's VASP and holder status must be active, and `targetAmount` must not exceed the receiver's limit. Otherwise it reverts with `HolderDoesNotExist`, `VaspNotActive`, `HolderNotActive`, or `ExceedsHolderLimit`.

A zero address on either side skips that side's checks, which is how mint (the `from` is zero) and burn (the `to` is zero) pass through. A transfer between two users is permitted only while both accounts and their VASPs are active, and the receiver stays within its holding limit.

3.3 Roles and Authority

Roles are stored in each contract's OpenZeppelin access-control state. CHFD and CHFD_VASP each define their own `DEFAULT_ADMIN_ROLE`.

- `DEFAULT_ADMIN_ROLE` (CHFD): Upgrades the token, sets the compliance contract address, and grants or revokes `MINTER_ROLE`, `BURNER_ROLE`, and `ENFORCEMENT_ROLE`, including through `grantMintBurnRoles()` and `revokeMintBurnRoles()`.
- `MINTER_ROLE` (CHFD): Mints through `mint()`.
- `BURNER_ROLE` (CHFD): Burns a holder that signed a permit, through `burnFromWithPermit()`.
- `ENFORCEMENT_ROLE` (CHFD): Burns any holder through `burnFrom()`, moves balances through `forceTransfer()`, and pauses or unpauses the token.
- `DEFAULT_ADMIN_ROLE` (CHFD_VASP): Upgrades the compliance contract and grants or revokes `UPDATE_VASP_ROLE` and `VALIDATE_TRANSFER_ROLE`.
- `UPDATE_VASP_ROLE` (CHFD_VASP): The relayer role. Creates VASPs, sets VASP status, and submits the signature-gated holder and admin operations.
- `VALIDATE_TRANSFER_ROLE` (CHFD_VASP): Held by the CHFD token contract. Authorizes calls to `validateTransfer()`.
- VASP admin: A per-VASP role stored in the VASP record. Governs holders within its own VASP.

The trust level and worst-case impact of each role are enumerated in the [Trust Model](#).

3.4 Deployment

Both contracts are deployed behind proxies and configured once. `CHFD.initialize()` grants the deployer `DEFAULT_ADMIN_ROLE`, `MINTER_ROLE`, `BURNER_ROLE`, and `ENFORCEMENT_ROLE`, sets the name and symbol, and records the compliance contract address. `CHFD_VASP.initialize()` grants the deployer `DEFAULT_ADMIN_ROLE` and `UPDATE_VASP_ROLE` and sets the EIP-712 domain.

Two steps complete the wiring between the contracts. The CHFD address must be granted `VALIDATE_TRANSFER_ROLE` on CHFD_VASP through `grantValidateTransferRole()`, without which every

transfer, mint, and burn reverts. The compliance contract address recorded on CHFD must point to the deployed CHFD_VASP.

After initialization the deployer holds every CHFD role and, on CHFD_VASP, `DEFAULT_ADMIN_ROLE` and `UPDATE_VASP_ROLE`. The deployer then distributes these roles to the designated holders. On each contract `DEFAULT_ADMIN_ROLE` is granted to two governance keys, a primary admin and a failover admin, so every contract carries two default admins that each hold full admin authority. `MINTER_ROLE`, `BURNER_ROLE`, and `UPDATE_VASP_ROLE` are granted together to the SSC relayer keys held in an HSM vault, and `ENFORCEMENT_ROLE` to a separate enforcement authority, kept apart from the operational roles by design. The holders of these roles are enumerated in the [Trust Model](#). Each role is revocable by the admin that administers it, and the deployer is assumed to revoke every role from itself at the end of deployment, retaining no capabilities. The CHFD and CHFD_VASP admin roles are independent on-chain, and the same primary and failover keys are assigned to both.

3.5 Upgradeability

Both contracts are UUPS-upgradeable. `_authorizeUpgrade()` is gated by `DEFAULT_ADMIN_ROLE` in each contract, thus the CHFD admin upgrades the token and the CHFD_VASP admin upgrades the compliance layer. Each contract disables initializers in its constructor and is configured once through `initialize()`. The `CHFD_V2` and `CHFD_VASP_V2` contracts are examples of a subsequent version. Each adds a `versionNumber` field set by a `reinitializer(2) initializeV2()`, a `version()` returning "V2", and a `setVersionNumber()` gated by `DEFAULT_ADMIN_ROLE`.

An upgrade is performed by deploying the new implementation contract and then calling `upgradeToAndCall()` on the proxy from a `DEFAULT_ADMIN_ROLE` holder, which routes through `_authorizeUpgrade()` for the role check and points the proxy at the new implementation. When the new version adds state, as `CHFD_V2` and `CHFD_VASP_V2` do, the same call passes encoded calldata for the `reinitializer(2) initializeV2()` so the implementation swap and the one-time state setup happen atomically in a single transaction.

3.6 Changes in Version 2

Version 2 introduces the following changes:

- `burnFrom()` and `burnFromWithPermit()` no longer call `validateTransfer()`. Each wraps `_burn()` in the force-transfer flag, so a burn succeeds regardless of the holder's or its VASP's status.
- `UPDATE_VASP_ROLE` can now remove holders with `removeHolder()`.
- `invalidateSignatureNonce()` was added. It lets a signer raise their own `nonces` value to a higher number, canceling their outstanding signatures.
- `vaspOwned` field was added to the `Holder` structure.
- `setHolderLimit()` and `setHolderStatus()` no longer require the holder's VASP to be active.

4 Trust Model

The CHFD system spans two upgradeable contracts. CHFD is the token itself, the ERC-20 that holders own and transfer. CHFD_VASP is the compliance layer. It holds VASP registration, holder allowlisting, limits, and status. It defines the transfer validation logic. Each contract anchors its roles in its own OpenZeppelin

`AccessControlUpgradeable` instance, thus each of them has a separate `DEFAULT_ADMIN_ROLE`.

Authority flows downward from these two roots of trust. The CHFD admin governs the token and appoints its operational roles (minter, burner, enforcer). The CHFD_VASP admin governs the compliance layer and appoints its operational roles (VASP updater, transfer validator). Both admins can replace their contract logic through UUPS upgrade. The two admin roles are independent on-chain, but as deployed the same primary key and the same failover key are assigned to both, so in practice the two roots are one party: control of an admin key is control of the whole system, not of a single contract. The SSC platform holds the operational keys (minter, burner, and VASP updater) behind the OpenZeppelin Relayer and Vault. The admin keys and the enforcement key are kept apart from the operational keys and assumed to be handled securely, held in secure physical custody such as a bank safe when not in use and managed under proper procedures when used. VASP organisations act through per-VASP admins authenticated by EIP-712 signatures. End customers act as registered holders.

Roles are listed below from the root of trust downward.

4.1 Roles and Permissions

4.1.1 CHFD Admin (`DEFAULT_ADMIN_ROLE` on `CHFD`)

- **Trust Level:** Fully trusted.
- **Permissions:** Root of trust for the token layer. Upgrades the token to arbitrary logic, repoints the compliance contract used for every transfer, mint, and burn check, and grants or revokes `MINTER_ROLE`, `BURNER_ROLE`, and `ENFORCEMENT_ROLE` to any address.
- **Worst-case:** Changes any token behavior, including every authorization and compliance check, by upgrading the implementation or repointing the validator to a permissive one. Appoints itself minter and enforcer, then mints to any holder and confiscates or burns any balance. Control over the token is unbounded.

4.1.2 CHFD VASP Admin (`DEFAULT_ADMIN_ROLE` on `CHFD_VASP`)

- **Trust Level:** Fully trusted.
- **Permissions:** Root of trust for the compliance layer. Upgrades the compliance contract to arbitrary logic and grants or revokes any role, including `UPDATE_VASP_ROLE` and `VALIDATE_TRANSFER_ROLE`.
- **Worst-case:** Rewrites all compliance logic through upgrade, including VASP and holder status, holder limits, and the transfer validation enforced on every transfer and mint. Appoints arbitrary VASP updaters. Neutralizing validation disables the entire compliance surface the token depends on. As deployed, the same keys hold this role and CHFD Admin, so a single admin key already has total control of the system, not just the compliance layer.

4.1.3 Enforcer (`ENFORCEMENT_ROLE` on `CHFD`)

- **Trust Level:** Fully trusted.
- **Permissions:** Regulatory intervention authority. Force-transfers tokens between any two addresses, bypassing the pause state and all VASP and holder validation. Burns from any address without an allowance or permit. Pauses and unpauses the token.

- **Worst-case:** Moves any holder's entire balance to an arbitrary account, including a non-holder, regardless of freeze, blocklist, or pause status, and burns any holder's balance. This is arbitrary confiscation and destruction of funds. Pausing denies all normal transfers, mints, and burns, leaving only force-transfer operational, and is recoverable through unpause.

4.1.4 Minter (`MINTER_ROLE` on `CHFD`)

- **Trust Level:** Trusted to mint only upon off-chain deposit.
- **Permissions:** Mints CHFD to any address, subject to the compliance check: the recipient must be a registered, active holder under an active VASP, and the resulting balance must not exceed the holder's configured limit. Typically held by the SSC relayer.
- **Worst-case:** Mints to every active holder up to that holder's configured limit. Issuance is bounded by the sum of holder limits and cannot exceed them by the minter alone. Minting to a holder the platform did not intend to credit is within the role's power.

4.1.5 Burner (`BURNER_ROLE` on `CHFD`)

- **Trust Level:** Trusted to burn only upon off-chain redeeming.
- **Permissions:** Burns tokens from a holder that produced an ERC-20 permit authorizing the burner as spender. Cannot burn without a valid permit. Typically held by the SSC relayer for redemption flows.
- **Worst-case:** Burns up to the permitted amount from any holder that signed a permit to the burner. Impact is limited to holders and the amounts they signed for.

4.1.6 VASP Updater (`UPDATE_VASP_ROLE` on `CHFD_VASP`)

- **Trust Level:** Fully Trusted.
- **Permissions:** Executes VASP and holder administration on behalf of the platform and the VASPs. Creates VASPs, gated by the prospective VASP admin's signature. Changes any VASP's status without a signature. Executes VASP-admin-signed operations over admins, holders, limits, and status. Typically held by the SSC relayer key.
- **Worst-case:** The only signature-free power is setting VASP status. Setting VASPs to any non-active state (`Blocked`, `Locked`, `Sanctioned`, `Frozen`) denies service to all their holders, because validation requires an active VASP, and applied to every VASP this halts all transfers and mints. All holder and admin mutations require a VASP-admin signature, so the VASP Updater alone cannot add holders, raise limits, or change holder status, and cannot mint, burn, or move funds. The denial of service is recoverable by re-activating the VASPs, by any VASP Updater or the admin. The case where this key also holds mint and burn roles is covered in Collusion.

4.1.7 Transfer Validator (`VALIDATE_TRANSFER_ROLE` on `CHFD_VASP`)

- **Trust Level:** Untrusted.
- **Permissions:** Gates `validateTransfer()`, granted to the CHFD token contract so it can query compliance during transfers, mints, and burns.
- **Worst-case:** The gated function is a read-only view and confers no state-changing power. The role only restricts who may invoke the check.

4.1.8 VASP Admin, per VASP

- **Trust Level:** Partially trusted. VASP admins are trusted to register only addresses they have previously KYC'd. This obligation is enforced off-chain through signed terms and conditions, user authentication, and an audit trail, not by the contract, which places no restriction on which addresses an admin may register or with what limit. VASPs trust that each other VASP follows those procedures. Each VASP is its own trust domain, and the model treats a VASP admin as fully trusted only over its own holders.

- **Permissions:** Governs holders within its own VASP only, by direct transaction or by EIP-712 signatures executed by the VASP updater. Adds and removes co-admins. Adds holders with arbitrary limits, and changes holder limits and status.
- **Worst-case:** Within its VASP, blocks, freezes, or sanctions holders, or sets a holder limit to 0, to deny them service, and registers arbitrary addresses as holders with arbitrary limits. Cannot mint, burn or move funds, so the impact is confined to its own holders. Raising a holder limit is consequential only in collusion with a minter, covered in Collusion.

4.1.9 Holder

- **Trust Level:** Untrusted.
- **Permissions:** A registered end-customer wallet bound permanently to one VASP. Holds, sends, and receives CHFD subject to compliance: the token must not be paused, both the holder and the VASP must be active, and the recipient balance must stay within its limit. Signs an ERC-20 permit to authorize the burn that settles a redemption.
- **Worst-case:** Moves its own balance to other registered holders within its limit and signs permits and consent payloads authorizing burns and VASP operations against itself. The impact is confined to its own funds and the operations it explicitly authorizes. It cannot exceed its limit, send to a non-holder, affect another holder, or call any privileged function.

4.1.10 End User, non-holder

- **Trust Level:** Untrusted.
- **Permissions:** None. An address that is not a registered holder cannot receive or hold CHFD, because validation reverts for unknown holders, and cannot call any privileged function.
- **Worst-case:** No capability beyond submitting transactions the contracts reject. Value can only reach a non-holder through a force-transfer, and once there it is stuck, since validation reverts for the non-holder it cannot be transferred out.

4.2 Collusion

The privileged roles are concentrated in one trust domain by design. Minter, Burner, and VASP Updater are the operational SSC roles, held by the relayer keys behind the Relayer and Vault. The Enforcer is SSC-held too, but kept apart from these operational keys, in secure custody with the admin keys rather than on the hot relayer path. All of them are SSC-held, whether or not SSC splits them across separate keys or teams, so their combination is the baseline trust assumption: the SSC operator is fully trusted and can mint, burn, pause, and upgrade, and these roles are not assumed to be mutually independent.

Since the SSC roles act as one party, only collusion with the independent VASP parties matters.

- **SSC operator and VASP admin:** A VASP admin that signs an inflated holder limit lets the operator mint beyond the platform's intended issuance.
- **VASP admin and holder, same VASP:** Stays within the VASP's own membership, limits, and status. Cannot mint, burn, or affect other VASPs.

Of these, only operator and VASP admin collusion escalates beyond an existing role's powers, so the residual trust assumption is that VASP admins do not collude with the SSC operator.

4.3 External Components

- **OpenZeppelin upgradeable contracts:** Access control, ERC-20, ERC-20 Permit, Pausable, UUPS proxy, EIP-712, and ECDSA recovery are inherited from OpenZeppelin. They are assumed to behave correctly and to be deployed behind a correctly configured ERC-1967 proxy.
- **OpenZeppelin Relayer, HashiCorp Vault, Azure HSM:** Off-chain custody and submission of the operational keys (minter, burner, VASP updater). The on-chain contracts trust whoever holds these roles, so the security of the operational surface reduces to the security of these key-management components, assumed to be configured per the platform's target hardening of Vault, HSM, and least-privilege policies.
- **Identity and orchestration (Keycloak, Temporal, SSC and VASP APIs):** These decide which operator may trigger a relayer transaction. They are outside the on-chain trust boundary. The contracts enforce only the role checks above and assume these systems gate access correctly.
- **EVM chain and RPC:** Execution, ordering, and finality are provided by the underlying EVM network and RPC endpoints, assumed to behave correctly within documented finality assumptions.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 Custom State Uses Sequential Storage Rather than ERC-7201

System Consideration	Version 1
----------------------	-----------

`CHFD` and `CHFD_VASP` are UUPS-upgradeable but declare their custom variables in plain sequential storage from slot 0, while the inherited OpenZeppelin contracts use ERC-7201 namespaced slots. This ties upgrade safety to careful management of the contract's sequential storage slots, which must only be appended to.

SC2 Holder Limit Caps Incoming Transfers, Not Standing Balance

System Consideration	Version 1
----------------------	-----------

The holder limit caps only incoming transfers: `validateAmount()` reverts when a recipient's post-transfer balance would exceed `holder.limit`. Because `setHolderLimit()` does not check the current balance, lowering a limit below a holder's existing balance leaves that balance untouched while blocking any further inflows. As a result, `balance <= limit` is not a maintained invariant.

SC3 Minting Is Constrained by the Recipient's Holder Limit

System Consideration	Version 1
----------------------	-----------

`CHFD.mint` calls `validateTransfer(address(0), to, balanceOf(to) + amount)`, which reverts if `balanceOf(to) + amount` exceeds the recipient's holder limit. A mint therefore succeeds only while the recipient stays within its limit.

Because the check reads the recipient's live balance, anyone can influence it by transferring tokens to the recipient. When a mint of `amount` would land exactly on the limit (the check is a strict `>`), a 1-wei transfer to the recipient tips `balanceOf(to) + amount` over the limit and reverts the mint until the limit is raised or the balance drops.

SC4 Proxy Deployment and Initialization Must Be Atomic

System Consideration	Version 1
----------------------	-----------

`CHFD` and `CHFD_VASP` are deployed behind proxies and configured once through `initialize()`, which grants the caller `DEFAULT_ADMIN_ROLE`. Because the constructors disable initializers on the implementation but leave the proxy uninitialized, the proxy's `initialize()` is callable by anyone until it runs.

If proxy deployment and `initialize()` are not bundled into a single atomic transaction, an attacker can frontrun `initialize()`, claim `DEFAULT_ADMIN_ROLE`, and, since each contract is UUPS-upgradeable with `_authorizeUpgrade()` gated by that role, upgrade the implementation to a malicious one. This is a Clandestine Proxy In The Middle of Proxy (CPIMP) attack, in which the substituted implementation acts as a man in the middle on every subsequent call.

Deployment must therefore deploy the proxy and call `initialize()` atomically.

SC5 `referenceId` Is Opaque on-Chain and Only as Trustworthy as Its Submitter

System Consideration	Version 1
<code>referenceId</code> is an off-chain correlation id emitted in events. The contract accepts any <code>bytes32</code>, never checks it, and never signs it, so its trust depends on the submitter. SSC-submitted operations carry the real id. The five direct VASP-admin functions (<code>addVaspAdmin</code>, <code>removeVaspAdmin</code>, <code>addHolder</code>, <code>setHolderLimit</code>, <code>setHolderStatus</code>) carry whatever the admin chooses: empty, reused, junk, or picked to collide with a real id. Off-chain consumers correlating on <code>referenceId</code> should treat values from direct VASP-admin operations as untrusted and not assume uniqueness or validity.	

SC6 Restricted Holder and VASP Statuses Are Equivalent on-Chain

System Consideration	Version 1
The <code>HolderStatus</code> and <code>VaspStatus</code> enums in <code>src/Statuses.sol</code> each define five non-<code>None</code> states: <code>Active</code>, <code>Blocked</code>, <code>Locked</code>, <code>Sanctioned</code>, and <code>Frozen</code>. On-chain only <code>Active</code> is distinguished: <code>validateStatus</code> and <code>validateAmount</code> revert whenever a holder or its VASP is not <code>Active</code>, and <code>Blocked</code>, <code>Locked</code>, <code>Sanctioned</code>, and <code>Frozen</code> are never referenced anywhere in the logic. The four restricted states therefore carry no distinct on-chain behavior and exist purely as labels interpreted off-chain.	

SC7 Signature Verification Accepts EOA Signers Only

System Consideration	Version 1
The <code>*WithSig</code> flows in <code>src/CHFD_VASP.sol</code> verify signatures exclusively through <code>ECDSA.recover</code> in <code>_verifySignature</code> and <code>_verifyVaspAdminSignature</code>, which recovers an EOA address from <code>(v, r, s)</code>. Smart-contract wallets that authenticate via EIP-1271 <code>isValidSignature</code> (for example Safe and other multisigs) cannot produce a recoverable signature and therefore cannot act as signing VASP admins.	

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

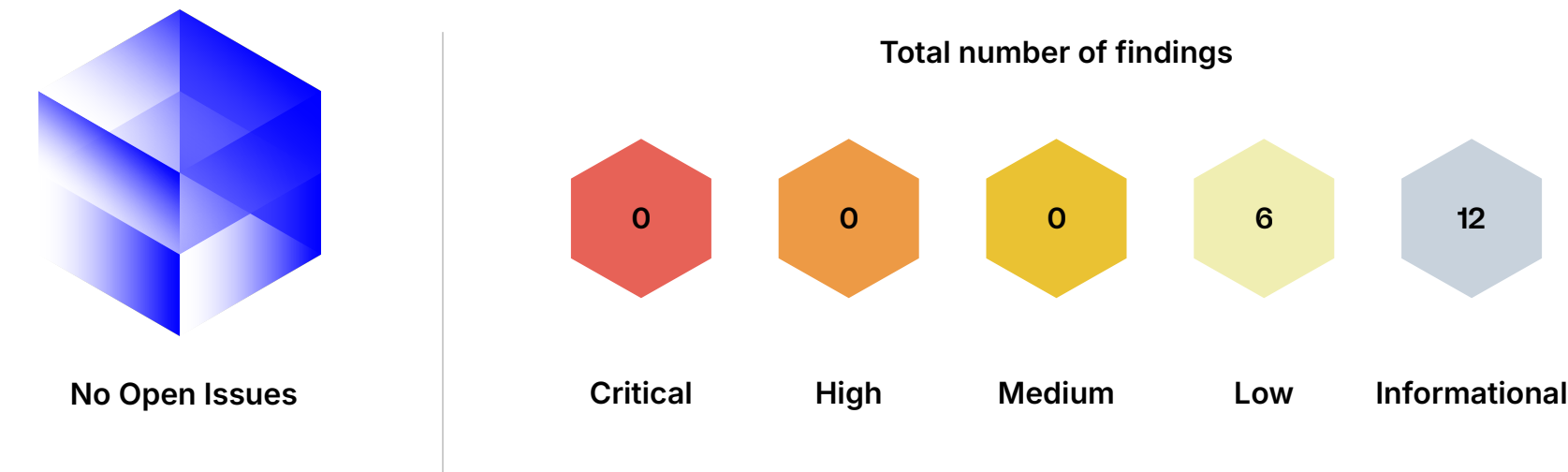
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Non-informational findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

Low	#001	Burn Paths Require an Active Holder and VASP and an Unpaused Token	Code Corrected	✓
Low	#002	burnFromWithPermit Can Be Griefed by Permit Front-Running	Code Corrected	✓
Low	#003	forceTransfer Can Send Funds to an Address That Is Not a Holder	Code Corrected	✓
Low	#004	Holder Limit and Status Updates Require the VASP to Be Active	Code Corrected	✓
Low	#006	Self-Transfer Double-Counts the Balance in the Holder Limit Check	Code Corrected	✓
Low	#007	Signers Cannot Cancel Issued Signatures Prior to Deadline Expiry	Code Corrected	✓
Info	#005	Role Concentration and Lack of Recovery Levers	Acknowledged	⊖
Info	#008	versionNumber Is an Unreliable and Redundant Version Indicator	Code Corrected	✓
Info	#009	A VASP Admin Can Create Holders Bypassing the addHolder Guard	Code Corrected	✓
Info	#010	Documentation Inconsistencies	Code Corrected	✓
Info	#011	Events Do Not Index All Filterable Fields	Code Corrected	✓
Info	#012	Gas Optimizations	Code Part. Corrected / Acknowledged	Ⓛ
Info	#013	Inconsistent Domain Separator Accessor Name	Code Corrected	✓
Info	#014	Misleading Getter Names	Specification Changed	✓
Info	#015	No Removal or Recovery Path for Mistakenly Added Holders	Code Corrected	✓
Info	#016	Outdated Compiler Version	Code Corrected	✓
Info	#017	Permit Domain Name Uses the Token Symbol	Code Corrected	✓
Info	#018	Single VASP Admin Can Capture a VASP with No Recovery Path	Acknowledged	⊖

7.2 Descriptions

#001 Burn Paths Require an Active Holder and VASP and an Unpaused Token

Design	Low	Version 1	Code Corrected ✓
--------	-----	-----------	------------------

`burnFrom` and `burnFromWithPermit` in `src/CHFD.sol` call `validateTransfer(from, address(0), 0)`, which runs `validateStatus(from)` and reverts unless the holder and its VASP are `Active`. Burning is therefore impossible for `Blocked`, `Locked`, `Sanctioned`, or `Frozen` holders, the exact accounts an enforcement action targets.

This matters for `burnFrom ( ENFORCEMENT_ROLE )`, which burns directly and is meant to remove funds from a restricted holder, yet is disabled for exactly those holders.

Both paths also route through `_update`, which calls `_requireNotPaused()` when the force-transfer flag is unset, so burning is blocked while the token is paused. Note that the `ENFORCEMENT_ROLE` can in any case force-move the balance with `forceTransfer` (which skips both the pause check and VASP validation) to an `Active` address and burn it there, so this is extra friction rather than a hard blocker.

Code Corrected

The `validateTransfer` call was removed from `burnFrom` and `burnFromWithPermit`. Both now wrap `_burn` in `_beginForceTransfer()` / `_endForceTransfer()`, bypassing the status and pause checks. The specification was updated to match this behavior.

#002 burnFromWithPermit Can Be Griefed by Permit Front-Running

Design	Low	Version 1	Code Corrected ✓
--------	-----	-----------	------------------

`burnFromWithPermit` in `src/CHFD.sol` calls `permit(owner, _msgSender(), amount, deadline, v, r, s)` internally, consuming the owner's ERC-2612 nonce. Because the signed permit is visible in the mempool, anyone can submit the same `permit` first. When the relayer's `burnFromWithPermit` then executes, the inner `permit` call reverts since the nonce is already used, and the burn fails until a fresh permit is signed.

Code Corrected

`burnFromWithPermit` now tolerates permit nonce front-running. The `permit` call is attempted only when the existing allowance is insufficient and is wrapped in a `try/catch`, so a front-run permit no longer reverts the burn. The authoritative authorisation check is `_spendAllowance(owner, _msgSender(), amount)`, which succeeds against any sufficient allowance regardless of whether the permit was consumed by the relayer or by a front-runner.

#003 forceTransfer Can Send Funds to an Address That Is Not a Holder

Design	Low	Version 1	Code Corrected ✓
--------	-----	-----------	------------------

`CHFD.forceTransfer` sets `_forceTransferInProgress`, which makes `_update` skip `validateTransfer`. So the recipient does not have to be a registered holder. If the recipient is not a holder, it cannot move the tokens afterwards: a normal transfer calls `validateTransfer`, which reverts with `HolderDoesNotExist` when the sender is not a holder.

The funds are not lost, because `ENFORCEMENT_ROLE` can call `forceTransfer` again to move them out. But until then they sit at an address that cannot spend them.

Code Corrected

`forceTransfer` now resolves the recipient's VASP id via `getHolderVaspId(to)` and reverts with `HolderDoesNotExist` when it is `bytes32(0)`. Funds can therefore no longer be force-moved into an unregistered address that could not subsequently spend them.

#004 Holder Limit and Status Updates Require the VASP to Be Active

Design	Low	Version 1	Code Corrected ✓
--------	-----	-----------	------------------

`_setHolderLimit()` and `_setHolderStatus()` both revert with `VaspNotActive` when the holder's VASP is not `Active`. So while a VASP is suspended (`Blocked`, `Locked`, `Sanctioned`, or `Frozen`), an operator cannot adjust an individual holder's limit or change a single holder's status.

For example, if a VASP is `Blocked` for a limited time during an investigation and the operator meanwhile needs to ban one holder or tighten a holder's limit, neither action is possible until the VASP is `Active` again.

Code Corrected

The `VaspStatus.Active` requirement was removed from `_setHolderLimit` and `_setHolderStatus`. An operator can now adjust a single holder's limit or status while the VASP is suspended.

#006 Self-Transfer Double-Counts the Balance in the Holder Limit Check

Correctness	Low	Version 1	Code Corrected ✓
-------------	-----	-----------	------------------

`CHFD._update` computes `targetAmount = balanceOf(to) + amount` and passes it to `validateTransfer`, which enforces `targetAmount <= holder.limit` on the recipient. When `from == to`, `balanceOf(to)` already includes `amount`, so `targetAmount` overstates the true post-transfer balance, which is unchanged.

A holder whose balance is at or near `limit` therefore reverts with `ExceedsHolderLimit` on a self-transfer, even though the balance does not grow.

Code Corrected

`_update` now computes `targetAmount = from == to ? balanceOf(to) : balanceOf(to) + amount`. On a self-transfer the recipient balance is no longer double-counted, so a holder at or near its limit can self-transfer without a spurious `ExceedsHolderLimit` revert.

#007 Signers Cannot Cancel Issued Signatures Prior to Deadline Expiry

Design	Low	Version 1	Code Corrected ✓
--------	-----	-----------	------------------

In `CHFD_VASP`, the `*WithSig` operations and `addVasp()` bind a per-account sequential nonce from `_useSignatureNonce()` into the signed EIP-712 struct. The nonce advances only when a signature for the current value is consumed by a successful operation. A signer has no function to advance their own nonce or otherwise invalidate a signature they have already issued.

No operation consumes a nonce without also performing a state change. Advancing past nonce N, and thereby invalidating any signature bound to N, requires executing another genuine operation signed over N. A VASP admin who signs an operation such as `addHolderWithSig()` and then retracts the decision, or whose signed payload leaks, therefore cannot invalidate the signature on its own. It stays valid until the deadline passes or a relayer submits it, so the exposure window depends entirely on how far ahead the deadline is set off-chain.

Code Corrected

`invalidateSignatureNonce(uint256 newNonce)` was added which allows the caller to advance their own nonce to any strictly greater value, which invalidates every signature bound to a lower nonce. This new function does not implement any access restriction, therefore can be called by any account, including non-privileged users.

#005 Role Concentration and Lack of Recovery Levers

Informational

Version 1

Acknowledged 

Authority over CHFD concentrates in a small set of privileged keys. Their powers are broad and hard to reverse, and the protocol provides no in-protocol recovery or guardian mechanism. Several operational keys can unilaterally lock users out or freeze funds, and the only role to rotate a compromised privileged role is a `DEFAULT_ADMIN_ROLE`. Although the [Trust Model](#) assumes that all privileged roles are fully trusted and must be carefully protected, we highlight several potential privilege-concentration risks:

- `MINTER_ROLE` (`CHFD.mint`) has no on-chain mint cap beyond each recipient's holder limit, and those limits are themselves set by VASP admins or the relayer. There is no global supply cap or per-period limit enforced on-chain.
- `UPDATE_VASP_ROLE` does more than relay signed payloads. It can unilaterally `addVasp` (create VASPs) and `setVaspStatus` (suspend or activate any VASP with no signature). Suspending a VASP makes `validateTransfer` revert for all of that VASP's holders.
- `DEFAULT_ADMIN_ROLE` is the only key able to reverse a VASP capture or any other role misuse (via upgrade, `setVaspContractAddress`, or role re-grants), but is intended to be a secured, slow-to-use key. There is no faster, lower-privilege guardian to contain a compromised operational key.
- `ENFORCEMENT_ROLE` (`CHFD`) can `pause` and `unpause` the entire token, halting all user transfers, mints, and burns at once, and can `forceTransfer` or `burnFrom` any holder's balance. A compromised enforcement key can confiscate funds or freeze the whole system with no lever apart from `DEFAULT_ADMIN_ROLE` revoking its role.
- Privileged roles use plain `AccessControlUpgradeable`, not a two-step variant. Grants via `grantRole` take effect immediately with no accept handshake, timelock, or confirmation. Granting a role (including `DEFAULT_ADMIN_ROLE`) to a wrong or uncontrolled address is effective at once, and if the previous holder then renounces, the mistake is unrecoverable.

The net effect is that recovering from the compromise of a trusted role requires the `DEFAULT_ADMIN_ROLE` to intervene, as the system lacks lower-privileged or more rapidly acting roles that can perform recovery.

Acknowledged

Swiss Stablecoin acknowledges the concentration of privileges. The protocol will initially be deployed in a controlled sandbox environment. Operational key management and role separation will be reconsidered after the sandbox phase.

#008 `versionNumber` Is an Unreliable and Redundant Version Indicator

Informational

Version 1

Code Corrected

In CHFD_V2 and CHFD_VASP_V2, `initializeV2()` sets `versionNumber` to 2, while `setVersionNumber()` lets the default admin overwrite `versionNumber` with any value at any time. The stored `versionNumber` is therefore not a reliable indicator of the deployed implementation. A reader that trusts `versionNumber` to identify the contract version can be misled.

This `versionNumber` also duplicates state that the inherited `Initializable` already tracks: since `initializeV2()` uses `reinitializer(2)`, the `_initialized` slot is already set to 2 and, unlike `versionNumber`, can only advance monotonically and cannot be set arbitrarily. The custom variable thus adds a second, mutable source of information (though `Initializable` only exposes it via the internal `_getInitializedVersion()`).

Code Corrected

The manually managed `versionNumber` state and the `setVersionNumber()` setter were removed from both CHFD_V2 and CHFD_VASP_V2. `versionNumber()` now returns OpenZeppelin's initializer version via `_getInitializedVersion()`.

#009 A VASP Admin Can Create Holders Bypassing the `addHolder` Guard

Informational

Version 1

Code Corrected

`addHolder` reverts `VaspNotActive` unless the VASP is `Active`. However, a VASP admin can sidestep this guard through the admin-management path: `addVaspAdmin` is gated only by `_onlyVasp` and `_onlyVaspAdmin` (no `Active` check), and `_addVaspAdmin` calls `_ensureHolder(vaspId, vaspAdminAddress, 0, ...)`, which registers a new `Active` holder. `removeVaspAdmin` then clears the admin flag but leaves that holder in place. Therefore, an admin can create a persistent holder for the VASP regardless of its status, bypassing the `VaspNotActive` guard. Impact is limited because such a holder is created with all limits set to 0, so it cannot meaningfully transact.

Code Corrected

`_addVaspAdmin` now reverts if the VASP is not `Active`. Thus, an admin can no longer create a persistent holder while the VASP is suspended, closing the bypass of the `addHolder` guard.

#010 Documentation Inconsistencies

Informational

Version 1

Code Corrected

The technical design document appears to be outdated and out of sync with the implemented contracts. A non-exhaustive list of examples:

- The document refers throughout to a `RELAYER_ROLE` (for `addVasp` and the `*WithSig` functions), but no such role exists in the code. These functions are gated by `UPDATE_VASP_ROLE`, and there is no separate relayer role distinct from the VASP updater.

- The document lists a `DOMAIN_SEPARATOR` accessor, while the contract exposes `domainSeparator` in `src/CHFD_VASP.sol`.
- The `mint` signature is shown as `mint(to, amount)`, omitting the `referenceId` parameter present in `CHFD.mint(address, uint256, bytes32)`.
- `burnFrom` is described as allowance-based and as calling `_spendAllowance`, but `burnFrom` (gated by `ENFORCEMENT_ROLE`) burns directly without touching allowance; only `burnFromWithPermit` spends an allowance.
- The relayed signature functions are documented without the `nonZero` modifier the code applies to `addVaspAdminWithSig`, `removeVaspAdminWithSig`, and `addHolderWithSig`.

Code Corrected

The documentation was updated to reflect the intended smart-contract functionality and a new `SPEC.md` was introduced as the official specification. The documented behaviour now matches the implementation.

#011 Events Do Not Index All Filterable Fields

Informational

Version 1

Code Corrected ✓

Events can mark up to three fields as `indexed`, which lets off-chain consumers filter logs by those fields. Several events in `Events.sol` use fewer than three indexed fields, yet leave fields that are natural filter keys non-indexed.

For example, `CHFDTransferEvent` (emitted by `_update()`) indexes `from` and `fromVaspId` but not `to` or `toVaspId`, using only two of three slots. Consumers cannot filter by recipient and must decode data fields instead. Indexing one of `to` or `toVaspId` restores symmetric filtering.

More broadly, indexing across `Events.sol` is inconsistent and worth reviewing so the fields most likely to be queried (addresses, VASP identifiers, reference identifiers) occupy the indexed slots.

Code Corrected

Event fields were re-indexed such that the query-relevant fields occupy the three available indexed slots.

#012 Gas Optimizations

Informational

Version 1

Code Partially Corrected / Acknowledged ⓘ

Find below a non-exhaustive list of gas optimizations across `src/CHFD.sol` and `src/CHFD_VASP.sol`.

- `_forceTransferInProgress` uses persistent storage (`SSTORE`) for a flag that lives only within a single `forceTransfer` call.
- The reentrancy check in `_beginForceTransfer` (`if (_forceTransferInProgress) revert ForceTransferReentry()`) is unreachable in the current implementation.
- On the force-transfer path, `_update` still computes `targetAmount = balanceOf(to) + amount` although `validateTransfer` is skipped.
- `grantWorkerRoles` / `revokeWorkerRoles` and the validate-transfer role wrappers duplicate the inherited `grantRole` / `revokeRole`.
- Several functions (`addHolder`, `setHolderLimit`, `setHolderStatus`, and their `WithSig` variants) check VASP/holder existence in their `onlyVasp` / `onlyHolder` modifiers, then the inner `_addHolder` /

`_setHolder*` re-check the same conditions more strictly (`status == Active` , belongs-to-VASP) and re-load the same `_vasps` / `_holders` slots, duplicating both the checks and the reads.

- `validateTransfer` 's existence check reads `_holders[...].vaspId` , then `validateStatus` / `validateAmount` read it again via a fresh `_holders` lookup rather than their local `holder` storage pointer.
- `mint` , `burnFrom` , `burnFromWithPermit` , and `_update` each make two external calls to the VASP contract: first `validateTransfer` , then a VASP-id getter (`getHolderVaspId` or `getTransferVaspIds`). The second call re-reads the same `_holders` entries the first already loaded.
- `validateTransfer` is gated by `onlyRole(VALIDATE_TRANSFER_ROLE)` , adding an access-control storage read to every mint, burn, and transfer. The function is a `view` , not modifying any state in the contract.

Code Partially Corrected

Two of the listed optimizations were applied:

- `validateTransfer` now loads each side's holder storage entry once (`fromHolder` , `toHolder`) and passes the storage pointer into `validateStatus` / `validateAmount` , removing the duplicate `_holders` lookups.
- `_update` now caches `_forceTransferInProgress` into a local once and only computes `targetAmount` inside the branch where validation actually runs, so the force-transfer path no longer computes a value it discards.

Acknowledged

The remaining items are acknowledged and intentionally retained.

#013 Inconsistent Domain Separator Accessor Name

Informational	Version 1	Code Corrected ✓
---------------	-----------	------------------

`CHFD_VASP` defines a custom public `domainSeparator()` returning `_domainSeparatorV4()` , while the token `CHFD` exposes the all-caps `DOMAIN_SEPARATOR()` inherited from `ERC20PermitUpgradeable` (the EIP-2612 permit convention). `domainSeparator()` is not a standard violation, just a non-standard getter whose lower-camel name is inconsistent with the token's `DOMAIN_SEPARATOR()` and with the casing tooling typically expects.

Code Corrected

`CHFD_VASP` now exposes the accessor as `DOMAIN_SEPARATOR()` , matching the all-caps EIP-2612 convention used by the token's inherited `DOMAIN_SEPARATOR()` . The accessor name is now consistent across both contracts.

#014 Misleading Getter Names

Informational	Version 1	Specification Changed ✓
---------------	-----------	-------------------------

Two getters in `src/CHFD_VASP.sol` have names that misdescribe their return values.

- `getVaspDetails(vaspId)` returns only the `VaspStatus` , not a set of details, which is inconsistent with `getHolderDetails` returning the limit, status, and VASP id.



- `getVaspAdminStatus(vaspId, memberAddress)` returns a `bool` indicating whether the address is a VASP admin, although the name suggests it returns a `VaspStatus`.

Specification Changed

The two getters `getVaspDetails` and `getVaspAdminStatus` were removed from `CHFD_VASP`.

#015 No Removal or Recovery Path for Mistakenly Added Holders

Informational

Version 1

Code Corrected ✓

The VASP registry exposes no way to remove a holder, so an address added by mistake (wrong holder address, typo, or incorrect recipient) stays bound to the VASP permanently. The same permanence affects VASP admins: `_addVaspAdmin` calls `_ensureHolder`, which registers the admin address as a holder, and although `removeVaspAdmin` clears the admin flag it leaves that holder entry intact.

The admin can still neutralize a wrong holder by setting its status to `Blocked` or `Frozen` via `setHolderStatus`, which makes it fail `validateTransfer`, but the binding itself cannot be cleared. Since `CHFD_VASP` is a UUPS proxy, recovery is only possible through a contract upgrade that adds a removal function.

Code Corrected

A `removeHolder(vaspId, holderAddress, referenceId)` function was added, callable by `UPDATE_VASP_ROLE`.

#016 Outdated Compiler Version

Informational

Version 1

Code Corrected ✓

The contracts use `pragma solidity ^0.8.30` and `foundry.toml` pins `0.8.30`. Solidity `0.8.34` fixes a high-severity IR code generator bug (present from `0.8.28` to `0.8.33`) that can swap `sstore` and `tstore` when clearing a persistent and a transient variable of the same type, but only under `--via-ir` with `delete` on transient storage. This code is unaffected: it does not use `--via-ir` and no `transient` storage. Should future changes enable `--via-ir` or introduce transient storage, a compiler version `>= 0.8.34` with a locked `pragma` avoids exposure to this bug.

Code Corrected

`foundry.toml` now pins `solc_version = "0.8.34"` and every source file uses the locked version `0.8.34`.

#017 Permit Domain Name Uses the Token Symbol

Informational

Version 1

Code Corrected ✓

`CHFD.initialize` calls `__ERC20Permit_init("CHFD")`, so the EIP-712 domain `name` used for `permit` and the domain separator is the token symbol `"CHFD"` rather than the ERC-20 name `"Swiss Stablecoin"` set in `__ERC20_init`. The OpenZeppelin convention is to initialize the permit domain with the token name. This is functionally harmless since the domain name is an arbitrary string, but off-chain code building permit signatures must use `"CHFD"`, not the token name, or recovery will fail.

Code Corrected

`CHFD.initialize` now calls `__ERC20Permit_init("Swiss Stablecoin")`, so the EIP-712 permit domain `name` follows the OpenZeppelin convention and matches the ERC-20 token name set in `__ERC20_init`.

#018 Single VASP Admin Can Capture a VASP with No Recovery Path

Informational Version 1 Acknowledged 

A single VASP admin can take unilateral control of any VASP. `addVaspAdmin` and `removeVaspAdmin (src/CHFD_VASP.sol)` are gated only by `onlyVaspAdmin`, so any one admin can add an attacker-controlled address and then remove every other admin. `removeVaspAdmin` reverts only when `adminCount == 1` (`CannotRemoveLastVaspAdmin`), which prevents an empty admin set but does not prevent reducing the set to a single attacker-controlled address or even a single EOA that no one controls.

There is no recovery path for a captured VASP. Neither `DEFAULT_ADMIN_ROLE` nor `UPDATE_VASP_ROLE` can reset a VASP's admin set: the relayed `addVaspAdminWithSig` and `removeVaspAdminWithSig` still require a current VASP admin's EIP-712 signature, and no admin-level override exists. Once a VASP's admins are replaced by addresses no one controls, the registry can only be repaired by upgrading the implementation.

Acknowledged

Swiss Stablecoin acknowledges the behavior and will consider adding functionality to recover from a malicious VASP admin in a future version of the contract.

8 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.